Exercícios de CRUD com Spring Boot e RESTful APIs

Exercício 01: Gerenciamento de Livros

Enunciado

Crie uma aplicação Spring Boot para gerenciar livros. O objetivo é praticar mapeamento JPA, criação de tabela customizada e uso do CRUD com operações RESTful.

Entidade: Livro

Atributo	Tipo	Descrição
id	Long	Identificador único (chave primária, auto-gerado)
titulo	String	Título do livro
autor	String	Autor do livro
anoPublicacao	Integer	Ano de publicação

Requisitos

1. Crie uma entidade JPA chamada `Livro` com os atributos especificados
2. Configure o nome da entidade JPA como `Livro`
3. Configure o nome da tabela no banco como `tb_livro`
4. Pelo menos dois atributos devem ter o nome do campo diferente do nome da coluna no banco, utilizando `@Column(name="...")`
5. Implemente um repositório Spring Data JPA para a entidade

6. Implemente um serviço para realizar as operações de CRUD
7. Crie um controlador REST que exponha os seguintes endpoints

Endpoints RESTful

- **POST** `/livros` — criar novo livro
- **GET** `/livros` — listar todos os livros
- **GET** `/livros/{id}` — buscar livro por ID
- **DELETE** `/livros/{id}` — deletar livro por ID

Observações

- Não é necessário implementar validação de dados
- Utilize as convenções Java para nomes de métodos, atributos e classes
- Seu código deve estar organizado em pacotes seguindo boas práticas (ex: entity, repository, service, controller)

Exercício 02: Gerenciamento de Produtos

Enunciado

Desenvolva uma API REST com Spring Boot para gerenciar produtos. O objetivo é praticar mapeamento JPA com tipos numéricos e operações CRUD completas.

Entidade: Produto

Atributo	Tipo	Descrição
id	Long	Identificador único (chave primária, auto-gerado)
nome	String	Nome do produto
preco	BigDecimal	Preço do produto
estoque	Integer	Quantidade em estoque

Requisitos

1. Crie uma entidade JPA chamada `Produto` com os atributos especificados
2. Configure o nome da entidade JPA como `Produto`
3. Configure o nome da tabela no banco como `tb_produto`
4. Pelo menos dois atributos devem ter o nome do campo diferente do nome da coluna no banco, utilizando `@Column(name="...")`
5. Implemente um repositório Spring Data JPA para a entidade
6. Implemente um serviço para realizar as operações de CRUD
7. Crie um controlador REST que exponha os seguintes endpoints

Endpoints RESTful

- **POST** `/produtos` — criar novo produto
- **GET** `/produtos` — listar todos os produtos
- **GET** `/produtos/{id}` — buscar produto por ID
- **DELETE** `/produtos/{id}` — deletar produto por ID

Observações

- Não é necessário implementar validação de dados
- Utilize as convenções Java para nomes de métodos, atributos e classes
- Seu código deve estar organizado em pacotes seguindo boas práticas

---

Exercício 03: Gerenciamento de Pedidos

Enunciado

Crie um serviço RESTful em Spring Boot para gerenciar pedidos. O objetivo é praticar mapeamento JPA com tipos de data/hora e operações CRUD.

Entidade: Pedido

Atributo	Tipo	Descrição
id	Long	Identificador único (chave primária, auto-gerado)
data	LocalDateTime	Data e hora do pedido
valorTotal	BigDecimal	Valor total do pedido
status	String	Status do pedido (ex: PENDENTE, CONFIRMADO, ENTREGUE)

Requisitos

1. Crie uma entidade JPA chamada `Pedido` com os atributos especificados
2. Configure o nome da entidade JPA como `Pedido`
3. Configure o nome da tabela no banco como `tb_pedido`
4. Pelo menos dois atributos devem ter o nome do campo diferente do nome da coluna no banco, utilizando `@Column(name="...")`
5. Implemente um repositório Spring Data JPA para a entidade
6. Implemente um serviço para realizar as operações de CRUD
7. Crie um controlador REST que exponha os seguintes endpoints

Endpoints RESTful

- **POST** `/pedidos` — criar novo pedido
- **GET** `/pedidos` — listar todos os pedidos
- **GET** `/pedidos/{id}` — buscar pedido por ID
- **DELETE** `/pedidos/{id}` — deletar pedido por ID

Observações

- Não é necessário implementar validação de dados
- Utilize as convenções Java para nomes de métodos, atributos e classes
- Seu código deve estar organizado em pacotes seguindo boas práticas

---

Exercício 04: Gerenciamento de Autores

Enunciado

Implemente uma aplicação Spring Boot para gerenciar autores. O objetivo é praticar mapeamento JPA com tipos de data e operações CRUD.

Entidade: Autor

Atributo	Tipo	Descrição
id	Long	Identificador único (chave primária, auto-gerado)
nome	String	Nome do autor
nacionalidade	String	Nacionalidade do autor
dataNascimento	LocalDate	Data de nascimento do autor

Requisitos

1. Crie uma entidade JPA chamada `Autor` com os atributos especificados
2. Configure o nome da entidade JPA como `Autor`
3. Configure o nome da tabela no banco como `tb_autor`
4. Pelo menos dois atributos devem ter o nome do campo diferente do nome da coluna no banco, utilizando `@Column(name="...")`
5. Implemente um repositório Spring Data JPA para a entidade
6. Implemente um serviço para realizar as operações de CRUD
7. Crie um controlador REST que exponha os seguintes endpoints

Endpoints RESTful

- **POST** `/autores` — criar novo autor
- **GET** `/autores` — listar todos os autores
- **GET** `/autores/{id}` — buscar autor por ID

- **DELETE** `/autores/{id}` — deletar autor por ID

Observações

- Não é necessário implementar validação de dados
- Utilize as convenções Java para nomes de métodos, atributos e classes
- Seu código deve estar organizado em pacotes seguindo boas práticas

---

Exercício 05: Gerenciamento de Categorias

Enunciado

Desenvolva uma API REST com Spring Boot para gerenciar categorias. O objetivo é praticar mapeamento JPA básico e operações CRUD.

Entidade: Categoria

Atributo	Tipo	Descrição
id	Long	Identificador único (chave primária, auto-gerado)
nome	String	Nome da categoria
descricao	String	Descrição da categoria

Requisitos

1. Crie uma entidade JPA chamada `Categoria` com os atributos especificados
2. Configure o nome da entidade JPA como `Categoria`
3. Configure o nome da tabela no banco como `tb_categoria`
4. Pelo menos dois atributos devem ter o nome do campo diferente do nome da coluna no banco, utilizando `@Column(name="...")`
5. Implemente um repositório Spring Data JPA para a entidade
6. Implemente um serviço para realizar as operações de CRUD

7. Crie um controlador REST que exponha os seguintes endpoints

Endpoints RESTful

- **POST** `/categorias` — criar nova categoria
- **GET** `/categorias` — listar todas as categorias
- **GET** `/categorias/{id}` — buscar categoria por ID
- **DELETE** `/categorias/{id}` — deletar categoria por ID

Observações

- Não é necessário implementar validação de dados
- Utilize as convenções Java para nomes de métodos, atributos e classes
- Seu código deve estar organizado em pacotes seguindo boas práticas

---

Exercício 06: Gerenciamento de Clientes

Enunciado

Crie um serviço RESTful em Spring Boot para gerenciar clientes. O objetivo é praticar mapeamento JPA e operações CRUD com múltiplos campos de texto.

Entidade: Cliente

Atributo	Tipo	Descrição
id	Long	Identificador único (chave primária, auto-gerado)
nome	String	Nome do cliente
email	String	Email do cliente
telefone	String	Telefone do cliente

Requisitos

1. Crie uma entidade JPA chamada `Cliente` com os atributos especificados
2. Configure o nome da entidade JPA como `Cliente`
3. Configure o nome da tabela no banco como `tb_cliente`
4. Pelo menos dois atributos devem ter o nome do campo diferente do nome da coluna no banco, utilizando `@Column(name="...")`
5. Implemente um repositório Spring Data JPA para a entidade
6. Implemente um serviço para realizar as operações de CRUD
7. Crie um controlador REST que exponha os seguintes endpoints

Endpoints RESTful

- **POST** `/clientes` — criar novo cliente
- **GET** `/clientes` — listar todos os clientes
- **GET** `/clientes/{id}` — buscar cliente por ID
- **DELETE** `/clientes/{id}` — deletar cliente por ID

Observações

- Não é necessário implementar validação de dados
- Utilize as convenções Java para nomes de métodos, atributos e classes
- Seu código deve estar organizado em pacotes seguindo boas práticas

---

Exercício 07: Gerenciamento de Fornecedores

Enunciado

Implemente uma aplicação Spring Boot para gerenciar fornecedores. O objetivo é praticar mapeamento JPA com campos específicos de negócio e operações CRUD.

Entidade: Fornecedor

Atributo	Tipo	Descrição
id	Long	Identificador único (chave primária, auto-gerado)
nomeFantasia	String	Nome fantasia do fornecedor
cnpj	String	CNPJ do fornecedor
contato	String	Contato do fornecedor

Requisitos

1. Crie uma entidade JPA chamada `Fornecedor` com os atributos especificados
2. Configure o nome da entidade JPA como `Fornecedor`
3. Configure o nome da tabela no banco como `tb_fornecedor`
4. Pelo menos dois atributos devem ter o nome do campo diferente do nome da coluna no banco, utilizando `@Column(name="...")`
5. Implemente um repositório Spring Data JPA para a entidade
6. Implemente um serviço para realizar as operações de CRUD
7. Crie um controlador REST que exponha os seguintes endpoints

Endpoints RESTful

- **POST** `/fornecedores` — criar novo fornecedor
- **GET** `/fornecedores` — listar todos os fornecedores
- **GET** `/fornecedores/{id}` — buscar fornecedor por ID
- **DELETE** `/fornecedores/{id}` — deletar fornecedor por ID

Observações

- Não é necessário implementar validação de dados
- Utilize as convenções Java para nomes de métodos, atributos e classes
- Seu código deve estar organizado em pacotes seguindo boas práticas

---

Exercício 08: Gerenciamento de Departamentos

Enunciado

Desenvolva uma API REST com Spring Boot para gerenciar departamentos. O objetivo é praticar mapeamento JPA e operações CRUD em contexto organizacional.

Entidade: Departamento

Atributo	Tipo	Descrição
id	Long	Identificador único (chave primária, auto-gerado)
nome	String	Nome do departamento
localizacao	String	Localização do departamento

Requisitos

1. Crie uma entidade JPA chamada `Departamento` com os atributos especificados
2. Configure o nome da entidade JPA como `Departamento`
3. Configure o nome da tabela no banco como `tb_departamento`
4. Pelo menos dois atributos devem ter o nome do campo diferente do nome da coluna no banco, utilizando `@Column(name="...")`
5. Implemente um repositório Spring Data JPA para a entidade
6. Implemente um serviço para realizar as operações de CRUD
7. Crie um controlador REST que exponha os seguintes endpoints

Endpoints RESTful

- **POST** `/departamentos` — criar novo departamento
- **GET** `/departamentos` — listar todos os departamentos
- **GET** `/departamentos/{id}` — buscar departamento por ID
- **DELETE** `/departamentos/{id}` — deletar departamento por ID

Observações

- Não é necessário implementar validação de dados
- Utilize as convenções Java para nomes de métodos, atributos e classes
- Seu código deve estar organizado em pacotes seguindo boas práticas

---

Exercício 09: Gerenciamento de Projetos

Enunciado

Crie um serviço RESTful em Spring Boot para gerenciar projetos. O objetivo é praticar mapeamento JPA com tipos de data e operações CRUD.

Entidade: Projeto

Atributo	Tipo	Descrição
id	Long	Identificador único (chave primária, auto-gerado)
nome	String	Nome do projeto
dataInicio	LocalDate	Data de início do projeto
dataFim	LocalDate	Data de término do projeto

Requisitos

1. Crie uma entidade JPA chamada `Projeto` com os atributos especificados
2. Configure o nome da entidade JPA como `Projeto`
3. Configure o nome da tabela no banco como `tb_projeto`
4. Pelo menos dois atributos devem ter o nome do campo diferente do nome da coluna no banco, utilizando `@Column(name="...")`
5. Implemente um repositório Spring Data JPA para a entidade
6. Implemente um serviço para realizar as operações de CRUD
7. Crie um controlador REST que exponha os seguintes endpoints

Endpoints RESTful

- **POST** `/projetos` — criar novo projeto
- **GET** `/projetos` — listar todos os projetos
- **GET** `/projetos/{id}` — buscar projeto por ID
- **DELETE** `/projetos/{id}` — deletar projeto por ID

Observações

- Não é necessário implementar validação de dados
- Utilize as convenções Java para nomes de métodos, atributos e classes
- Seu código deve estar organizado em pacotes seguindo boas práticas

---

Exercício 10: Gerenciamento de Tarefas

Enunciado

Implemente uma aplicação Spring Boot para gerenciar tarefas. O objetivo é praticar mapeamento JPA com tipos de data e booleanos, e operações CRUD.

Entidade: Tarefa

Atributo	Tipo	Descrição
id	Long	Identificador único (chave primária, auto-gerado)
descricao	String	Descrição da tarefa
dataVencimento	LocalDate	Data de vencimento da tarefa
concluida	Boolean	Status de conclusão da tarefa

Requisitos

1. Crie uma entidade JPA chamada `Tarefa` com os atributos especificados

2. Configure o nome da entidade JPA como `Tarefa`
3. Configure o nome da tabela no banco como `tb_tarefa`
4. Pelo menos dois atributos devem ter o nome do campo diferente do nome da coluna no banco, utilizando `@Column(name="...")`
5. Implemente um repositório Spring Data JPA para a entidade
6. Implemente um serviço para realizar as operações de CRUD
7. Crie um controlador REST que exponha os seguintes endpoints

Endpoints RESTful

- **POST** `/tarefas` — criar nova tarefa
- **GET** `/tarefas` — listar todas as tarefas
- **GET** `/tarefas/{id}` — buscar tarefa por ID
- **DELETE** `/tarefas/{id}` — deletar tarefa por ID

Observações

- Não é necessário implementar validação de dados
- Utilize as convenções Java para nomes de métodos, atributos e classes
- Seu código deve estar organizado em pacotes seguindo boas práticas

---

Estrutura Recomendada do Projeto

```
src/main/java/com/seu/pacote/
├── entity/
│   ├── Livro.java
│   ├── Produto.java
│   ├── Pedido.java
│   ├── Autor.java
│   ├── Categoria.java
│   ├── Cliente.java
│   ├── Fornecedor.java
│   ├── Departamento.java
│   ├── Projeto.java
│   └── Tarefa.java
├── repository/
│   ├── LivroRepository.java
│   ├── ProdutoRepository.java
│   ├── PedidoRepository.java
│   ├── AutorRepository.java
│   ├── CategoriaRepository.java
│   ├── ClienteRepository.java
│   ├── FornecedorRepository.java
│   ├── DepartamentoRepository.java
│   ├── ProjetoRepository.java
│   └── TarefaRepository.java
├── service/
│   ├── LivroService.java
│   ├── ProdutoService.java
│   ├── PedidoService.java
│   ├── AutorService.java
│   ├── CategoriaService.java
│   ├── ClienteService.java
│   ├── FornecedorService.java
│   ├── DepartamentoService.java
│   ├── ProjetoService.java
│   └── TarefaService.java
├── controller/
│   ├── LivroController.java
│   ├── ProdutoController.java
│   ├── PedidoController.java
│   ├── AutorController.java
│   ├── CategoriaController.java
│   ├── ClienteController.java
│   ├── FornecedorController.java
```

```
│   ├── DepartamentoController.java
│   ├── ProjetoController.java
│   └── TarefaController.java
└── Application.java
```

Boas Práticas

1. **Organização em Pacotes**: Mantenha o código organizado em pacotes específicos (entity, repository, service, controller)
2. **Convenções de Nomenclatura**: Use nomes descritivos e siga as convenções Java
3. **Anotações JPA**: Utilize corretamente as anotações `@Entity`, `@Table`, `@Column`, `@Id`, `@GeneratedValue`
4. **Spring Data JPA**: Estenda `JpaRepository` para criar repositórios com operações CRUD automáticas
5. **Serviços**: Implemente a lógica de negócio na camada de serviço
6. **Controladores REST**: Exponha os endpoints RESTful com as anotações `@RestController`, `@RequestMapping`, `@PostMapping`, `@GetMapping`, `@DeleteMapping`
7. **HTTP Status**: Utilize os status HTTP apropriados (201 para criação, 200 para sucesso, 404 para não encontrado, 204 para deleção)

Documento de Exercícios de CRUD com Spring Boot e RESTful APIs

Todos os exercícios seguem o padrão de mapeamento JPA, criação de tabelas customizadas e operações CRUD com endpoints RESTful.